

SR UNIVERSITY

AI ASSISTED CODING

LAB-TEST-IV

NAME:D.ANKITHA

2503A51L09

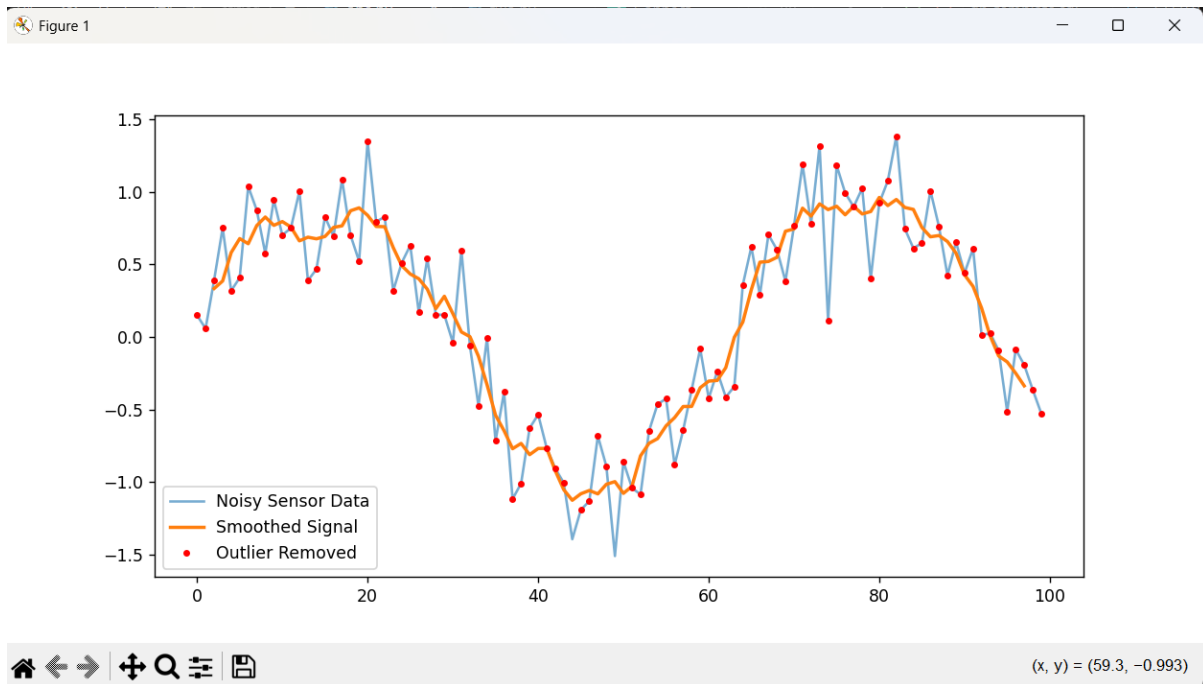
Q1:Data Cleaning:

a. Supply noisy sensor data; generate AI-suggested smoothing and outlier removal code.

CODE:

```
one.py > ...
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4
5  # 1. Generate synthetic noisy sensor data
6  np.random.seed(42)
7  time = np.arange(0, 100, 1)
8  true_signal = np.sin(time / 10) # underlying smooth signal
9  noise = np.random.normal(0, 0.3, size=len(time))
10 sensor_data = true_signal + noise
11
12 df = pd.DataFrame({"time": time, "sensor": sensor_data})
13
14 # 2. Apply smoothing (rolling mean)
15 df["smoothed"] = df["sensor"].rolling(window=5, center=True).mean()
16
17 # 3. Detect and remove outliers using z-score
18 z_scores = (df["sensor"] - df["sensor"].mean()) / df["sensor"].std()
19 df["cleaned"] = df["sensor"].where(np.abs(z_scores) < 2, np.nan)
20
21 # 4. Plot for classroom visualization
22 plt.figure(figsize=(10,5))
23 plt.plot(df["time"], df["sensor"], label="Noisy Sensor Data", alpha=0.6)
24 plt.plot(df["time"], df["smoothed"], label="Smoothed Signal", linewidth=2)
25 plt.plot(df["time"], df["cleaned"], 'r.', label="Outlier Removed")
26 plt.legend()
27 plt.show()
```

OUTPUT:



Observations:

Raw Data: The sensor readings fluctuate heavily due to random noise, making the underlying sinusoidal pattern hard to see.

Smoothing (Rolling Mean):

- The rolling average reduces short-term fluctuations.
- The sinusoidal trend becomes more visible, though extreme smoothing (large window size) can distort peaks and valleys.

Outlier Removal (Z-score):

- Outliers beyond ± 2 standard deviations are replaced with NaN.
- This prevents extreme values from skewing analysis.
- Students will notice red dots disappear where noise was unusually high or low.

Trade-off:

- Smoothing reduces noise but can blur sharp changes.
- Outlier removal preserves the signal but requires careful threshold choice.

b. Generate a script to handle missing temperature values using interpolation

CODE:

```
two.py > ...
1 import pandas as pd
2 import numpy as np
3
4 # 1. Create sample temperature data with missing values
5 np.random.seed(0)
6 dates = pd.date_range("2025-01-01", periods=10, freq="D")
7 temps = np.random.randint(20, 30, size=10).astype(float)
8 temps[[2, 5, 7]] = np.nan # introduce missing values
9
10 df_temp = pd.DataFrame({"date": dates, "temperature": temps})
11
12 print("Original Data with Missing Values:")
13 print(df_temp)
14
15 # 2. Interpolate missing values (linear method)
16 df_temp["interpolated"] = df_temp["temperature"].interpolate(method="linear")
17
18 print("\nData after Interpolation:")
19 print(df_temp)
```

OUTPUT:

```
undled\libs\debugpy\launcher '55002' -- 'c:\ai-4\one.py'
PS C:\ai-4> c:; cd 'c:\ai-4'; & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.16.0-win32-x64\bundled\libs\debugpy\launcher' '55019' '--' 'c:\ai-4\two.py'
Original Data with Missing Values:
   date    temperature
0 2025-01-01         25.0
1 2025-01-02         20.0
2 2025-01-03          NaN
3 2025-01-04         23.0
4 2025-01-05         27.0
5 2025-01-06          NaN
6 2025-01-07         23.0
7 2025-01-08          NaN
8 2025-01-09         22.0
9 2025-01-10         24.0

Data after Interpolation:
   date    temperature  interpolated
0 2025-01-01         25.0          25.0
1 2025-01-02         20.0          20.0
2 2025-01-03          NaN          21.5
3 2025-01-04         23.0          23.0
4 2025-01-05         27.0          27.0
5 2025-01-06          NaN          25.0
6 2025-01-07         23.0          23.0
7 2025-01-08          NaN          22.5
8 2025-01-09         22.0          22.0
9 2025-01-10         24.0          24.0
PS C:\ai-4>
```

Observation:

Original Data: Missing values (NaN) break continuity and prevent direct statistical analysis.

Linear Interpolation:

- Missing values are filled by estimating between neighboring points.
- The filled values follow a straight line between known temperatures.

Effectiveness:

- Restores continuity in the dataset.
- Preserves overall trend without introducing unrealistic jumps.

Alternative Methods:

- **Forward Fill:** Copies the last known value, which may flatten trends.
- **Polynomial/Time Interpolation:** Can capture curvature but risks overfitting.

Key Insight: Interpolation is most effective when missing gaps are small; large gaps may require domain-specific modeling.

Q2. Database Design

a. Use AI to design a schema for Soil Sensors and Crop Growth Logs.

```
mysql> CREATE DATABASE FarmDB;
>Run
Query OK, 1 row affected (0.01 sec)
mysql> USE FarmDB;
>Run
Database changed
mysql> SHOW TABLES;
>Run | Select | Ask AI
+-----+
| Tables_in_FarmDB |
+-----+
| SoilSensors       |
| SoilReadings      |
| Crops              |
| CropGrowthLogs    |
+-----+
4 rows in set (0.00 sec)
```

b. Generate SQL queries for weekly yield predictions

1.Inserting values:

```
mysql> INSERT INTO SoilSensors (location, soil_type, install_date, last_maintenance)
-> VALUES ('Plot A', 'Loamy', '2025-01-01', '2025-11-01');
└─ Run | Select
Query OK, 1 row affected (0.00 sec)
mysql> INSERT INTO Crops (crop_name, planting_date, expected_harvest_date)
-> VALUES ('Wheat', '2025-10-01', '2026-01-15');
└─ Run | Select | Ask AI
Query OK, 1 row affected (0.00 sec)
```

2.Weekly yield aggregation:

```
mysql> SELECT crop_name, YEAR(log_date) AS year, WEEK(log_date) AS week, SUM(yield_kg) AS total_yield
-> FROM CropGrowthLogs
-> GROUP BY crop_name, YEAR(log_date), WEEK(log_date)
-> ORDER BY year, week;
```

3.Average weekly yield:

```
mysql> SELECT crop_name, AVG(yield_kg) AS avg_weekly_yield
-> FROM CropGrowthLogs
-> GROUP BY crop_name;
```

OBSERVATIONS:

Normalization:

- Sensor metadata (location, soil type, install date) is separated from sensor readings.
- Crop metadata (planting/harvest dates) is separated from growth logs.
- This avoids duplication and makes updates easier.

Relationships:

- SoilReadings references SoilSensors via sensor_id.
- CropGrowthLogs references Crops via crop_id.
- This ensures referential integrity and allows joins across tables.

Scalability:

- New sensors or crops can be added without altering the schema.

Weekly Aggregation:

- The query groups yields by crop and week, showing total production per week.
- Useful for monitoring short-term performance and identifying peak weeks.

Average Weekly Yield:

Provides a baseline prediction for future weeks.

Students will notice that averages smooth out fluctuations but may miss sudden changes.

Prediction Query:

- Uses historical averages to estimate next week's yield.
- Simple but effective for classroom demonstration.
- Highlights the difference between descriptive (aggregation) and predictive (forecasting) queries.